AI/ML Engineering Portfolio: Architecture and Implementation Notes

Technical project portfolio

June 13, 2026

1 Overview

This repository contains three working software projects designed for technical portfolio review. Each folder is a runnable reference implementation with source code, sample data, tests, documentation, and deployment or monitoring configuration where appropriate.

1. **Local LLM Document Q&A System with Retrieval-Augmented Generation**: a local RAG system that indexes technical documents, retrieves relevant chunks, and answers questions with citations.
2. **ML Model Deployment Pipeline with CI/CD and Monitoring**: a PyTorch classifier served through FastAPI with model versioning, rollback, Prometheus metrics, Grafana dashboards, Docker Compose, and a GitHub Actions workflow.
3. **Real-Time Security Log Detection Pipeline**: a streaming-style SSH authentication log detector with parsing, brute-force detection, anomalous sign-in heuristics, Kafka-compatible streaming mode, an offline demo, metrics, and dashboards.

2 Project 1: Local RAG Document Q&A

2.1 Problem

The project answers questions over a private local document corpus without sending document content to an external hosted LLM. A standard RAG system must solve four subproblems: loading documents, chunking text, retrieving relevant context, and generating grounded answers.

2.2 Implemented Architecture

The folder `local-llm-document-qa-rag` contains the implementation. The main data flow is:

```
local documents -> loader -> overlapping chunks -> embeddings -> vector store
question -> query embedding -> top-k retrieval -> prompt/answer generator -> cited answer
```

Important files are:

- `src/rag_app/document_loader.py` loads Markdown, text, and optional PDF documents.
- `src/rag_app/chunker.py` creates overlapping word chunks with stable citation identifiers.
- `src/rag_app/embeddings.py` provides deterministic hashing embeddings by default and an optional sentence-transformer adapter.

- `src/rag_app/vector_store.py` provides a portable JSON vector store and an optional ChromaDB adapter.
- `src/rag_app/llm.py` calls Ollama when available and falls back to a deterministic extractive answerer.
- `src/rag_app/evaluate.py` measures retrieval relevance, keyword coverage, and an approximate hallucination rate.

2.3 Why the fallback design matters

A local LLM stack can be difficult to run on every laptop because Ollama models and ChromaDB require additional services and disk space. The implementation therefore has a fully functional dependency-light path: hashing embeddings, JSON persistence, and an extractive answerer. This makes the system testable immediately while still supporting the stronger target architecture through optional Ollama, sentence-transformers, and ChromaDB.

2.4 Setup and review commands

```
cd local-llm-document-qa-rag
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
python -m rag_app.cli ingest --docs data/docs --store .rag_store
python -m rag_app.cli ask --question "How are answers grounded?" --store .rag_store
python -m rag_app.evaluate --docs data/docs --store .rag_store --eval-file data/eval/questions
python -m unittest discover -s tests
```

2.5 Difficulty assessment

The basic version is moderate difficulty because it combines several straightforward components. The full version becomes harder when using real PDFs, GPU-backed embedding models, large vector stores, reranking, prompt evaluation, and hallucination analysis. The most error-prone parts are chunking strategy, retrieval quality, citation enforcement, and evaluation methodology.

3 Project 2: ML Deployment Pipeline

3.1 Problem

Training a model is only part of an ML system. A deployable service needs a stable prediction contract, versioned artifacts, automated tests, rollback, and runtime monitoring. This project demonstrates that end-to-end path using a small PyTorch tabular classifier.

3.2 Implemented Architecture

The folder `ml-model-deployment-pipeline` contains the implementation. The main data flow is:

```
synthetic training data -> PyTorch model -> versioned checkpoint -> registry
FastAPI request -> feature validation -> active model -> prediction -> metrics
```

Important files are:

- `src/ml_service/train.py` generates synthetic data and trains versioned PyTorch models.
- `src/ml_service/model.py` defines the neural network and prediction wrapper.
- `src/ml_service/features.py` defines the feature contract and ordering.
- `src/ml_service/registry.py` stores the active version and supports rollback.
- `src/ml_service/app.py` exposes FastAPI endpoints for health, prediction, model info, rollback, and metrics.
- `monitoring/` contains Prometheus and Grafana provisioning.
- `.github/workflows/ci.yml` defines a CI pipeline for tests, reproducibility checks, and Docker build.

3.3 Prediction service behavior

The API validates four numerical features: account age, failed-login count, transaction amount, and device trust score. The service normalizes features using training reference statistics stored in the checkpoint, runs the active PyTorch model, returns a risk probability and label, and records latency, throughput, prediction count, active version, and drift score.

3.4 Rollback behavior

The file-backed model registry stores an active version and deployment history. The endpoint `POST /admin/rollback` moves the active version back to the previous entry, reloads the model, and updates the active-model metric. This is a compact version of a deployment rollback mechanism used in real ML services.

3.5 Setup and review commands

```
cd ml-model-deployment-pipeline
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
python -m pytest
uvicorn ml_service.app:app --reload --port 8080
```

Example prediction request:

```
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"account_age_days":45,"failed_login_count":7,
       "transaction_amount":420.0,"device_trust_score":0.22}'
```

3.6 Difficulty assessment

This project is moderate to high difficulty because it crosses ML, backend engineering, DevOps, and observability. The model itself is simple. The more realistic engineering effort is in reproducible artifacts, schema stability, Docker packaging, metrics, dashboard provisioning, test coverage, and rollback semantics.

4 Project 3: Real-Time Security Log Detection Pipeline

4.1 Problem

Authentication logs are noisy and high volume. A useful detection pipeline must parse inconsistent text logs, maintain time-windowed state, detect suspicious patterns, and surface alerts quickly. This project implements common SSH detection rules and provides both offline and Kafka-compatible streaming modes.

4.2 Implemented Architecture

The folder `real-time-security-log-detection-pipeline` contains the implementation. The main data flow is:

```
auth.log lines -> parser -> AuthEvent objects -> detection engine -> Alert objects
Kafka mode: producer -> auth-logs topic -> detector consumer -> security-alerts topic
```

Important files are:

- `src/security_pipeline/log_parser.py` parses common OpenSSH authentication events.
- `src/security_pipeline/detector.py` implements sliding-window detection rules.
- `src/security_pipeline/offline_demo.py` runs the pipeline without Kafka for fast review.
- `src/security_pipeline/producer.py` replays parsed events into Kafka.
- `src/security_pipeline/consumer.py` consumes events, emits alerts, and exposes metrics.
- `monitoring/` contains Prometheus and Grafana configuration.
- `data/sample_auth.log` contains sample events that trigger expected alerts.

4.3 Detection logic

The detector uses three practical rules:

- **Brute force**: multiple failed SSH authentication attempts for the same user and source IP in a short window.
- **New device login**: a user with a known successful-login source IP authenticates from a new source IP.
- **Successful-login velocity**: a user authenticates successfully from several distinct source IPs inside a short time window.

The engine keeps bounded deques for recent failures and successes, so memory use stays controlled during streaming. Alert cooldowns prevent repeated alerts for the same condition from flooding downstream systems.

4.4 Setup and review commands

```
cd real-time-security-log-detection-pipeline
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
python -m security_pipeline.offline_demo --input data/sample_auth.log --output alerts.jsonl
python -m unittest discover -s tests
```

For streaming mode:

```
docker compose up --build
```

4.5 Difficulty assessment

The offline rule engine is moderate difficulty. The streaming version is higher difficulty because Kafka-style systems introduce broker configuration, consumer groups, serialization, retry behavior, state management, monitoring, and deployment concerns. Real production versions would also need log normalization for more formats, IP reputation enrichment, user baselines, persistence for state, and integration with SIEM or incident-response tools.

5 Testing Summary

Each project includes tests that exercise the core behavior.

Project	Tested behavior
Local RAG	Chunking overlap, invalid chunk parameters, ingestion, retrieval, cited answer generation, hallucination-rate helper.
ML Deployment	Feature ordering, registry rollback, FastAPI health endpoint, and prediction endpoint.
Security Pipeline	SSH log parsing, brute-force detection, new-device detection, velocity detection, and offline alert generation.

A root convenience script is also included:

```
./run_all_tests.sh
```

6 Overall Build Difficulty

The three projects become progressively more complex as they integrate more infrastructure. The RAG project teaches document processing and retrieval quality. The ML deployment project teaches service contracts, model artifacts, and monitoring. The security pipeline teaches streaming state, detection rules, and operational dashboards. A polished version of all three is feasible for an experienced student or junior engineer, and each project has production-level extension paths that can become substantial engineering efforts.

7 Production Hardening Ideas

- Add authentication and authorization to APIs and dashboards.
- Add persistent databases for audit logs, model metadata, and detection state.
- Add structured logging and trace IDs across services.
- Add stronger CI checks, vulnerability scanning, and deployment environments.
- Add larger evaluation datasets and stronger metrics for RAG faithfulness and ML drift.
- Add alert routing to email, Slack, PagerDuty, or a SIEM platform.